

Institut Sa Palomera . ASIX

Hospital Management System

Sistema de Gestio Hospitalaria per a l'Hospital de Blanes

Projecte Intermodular

Autor: Yossef Errazik

Curs: 2025-2026

Data: Maig de 2026

<https://github.com/yosseferrazik/hospital-management-project>


```
body { font-family: 'Segoe UI', Arial, sans-serif; font-size: 11pt; line-height: 1.6; color: #222; margin: 0; padding: 20pt; }
h1 { font-size: 20pt; color: #1a5276; border-bottom: 2px solid #1a5276; padding-bottom: 4pt; margin-top: 24pt;
page-break-before: auto; }
h2 { font-size: 16pt; color: #2e86c1; margin-top: 20pt; }
h3 { font-size: 13pt; color: #2980b9; margin-top: 16pt; }
table { border-collapse: collapse; width: 100%; margin: 12pt 0; font-size: 9pt; }
th, td { border: 1px solid #999; padding: 3pt 6pt; text-align: left; }
th { background: #1a5276; color: white; }
tr:nth-child(even) td { background: #f0f4f8; }
code { background: #e8e8e8; padding: 1pt 3pt; border-radius: 2pt; font-size: 9pt; }
pre { background: #f4f4f4; border: 1px solid #ddd; padding: 8pt; font-size: 9pt; overflow-x: auto; }
pre code { background: none; padding: 0; }
img { max-width: 95%; height: auto; margin: 6pt 0; }
blockquote { background: #eaf2f8; border-left: 4pt solid #2980b9; margin: 10pt 0; padding: 6pt 10pt; font-style: italic; }
.cover-page { text-align: center; padding-top: 100pt; }
.cover-page h1 { font-size: 28pt; color: #1a5276; border: none; margin-bottom: 10pt; }
.cover-page h2 { font-size: 16pt; color: #555; font-weight: normal; border: none; }
.cover-page p { font-size: 11pt; color: #555; margin: 3pt 0; }
.cover-institution { font-size: 13pt; color: #888; margin-bottom: 30pt; }
@media print { .no-break { page-break-inside: avoid; } }
```

Institut Sa Palomera · ASIX

Projecte Intermodular

Autor: Yossef Errazik

Curs: 2025–2026

Data: Maig de 2026

Repositori: github.com/yosseferrazik/hospital-management-project

CAT: Aquest document constitueix el lliurable final del projecte Hospital Management System (HMS), desenvolupat per a l'Hospital de Blanes. Recull de manera sintetitzada tots els apartats del projecte, incloent-hi la planificació, el disseny de la base de dades, la implementació de l'API i el client d'escriptori, la seguretat, l'alta disponibilitat, les consultes, l'exportació de dades i la documentació tècnica completa.

ENG: This document is the final deliverable of the Hospital Management System (HMS) project, developed for Hospital

de Blanes. It synthesises all project sections including planning, database design, API and desktop client implementation, security, high availability, queries, data export, and complete technical documentation.

Tkinter Desktop Client —HTTP— Flask API —SQL— PostgreSQL 16

Capa / Layer

Tecnologia / Technology

Versió

Backend

Python + Flask

3.13 / 3.1

API Server

Gunicorn (4 workers) + systemd

23.0

ORM

SQLAlchemy

2.0

Base de dades

PostgreSQL

16

Autenticació

JWT (Flask-JWT-Extended) + bcrypt

4.7 / 5.0

Client

Tkinter

—

Dashboard

Chart.js

—

Dades falses

Faker (es_ES, ru_RU)

40.15

Xarxa

Tailscale VPN

Node

Ubicació

IP Tailscale

Rol

Briar

Sala de servidors hospital

100.78.155.2

API + PostgreSQL primari

Sion

AWS EC2 (eu-west-3, París)

100.98.214.53

PostgreSQL standby (read-only)

Hospital users connect to Briar at 192.168.4.254 on the LAN. Admin connects via Tailscale.

[Image:  C:\Users\yossef\Proyectos y actividades\Proyectos Intermodulares\hospital-management-project\docs\01_planning\images\C4_Context_Diagram.drawio.png]

Aquesta secció està redactada en català.

Membre

Rol

Yossef Errazik

Desenvolupament complet

GitHub: github.com/yosseferrazik/hospital-management-project

Tecnologies escollides:

Capa

Tecnologia

Per què

Backend

Python 3.12 + Flask 3.1

Ja el coneixíem de classe, zero cost de llicència

Base de dades

PostgreSQL 16

Complir requisits (cifrall·lic, SSL, replicació)

ORM

SQLAlchemy 2.0

Prevenir SQL injection, facilitar manteniment

Client

Tkinter

Va inclòs amb Python, funciona en equips modestos

Dashboard

Chart.js

Alternativa gratuïta a PowerBI

Autenticació

JWT + bcrypt

Estàteless, suficient per a ús intern

Virtualització

Tailscale

Connectar els dos nodes sense obrir ports públics

Planificació de tasques (estimades vs. reals):

Tasca

Inici

Fi

H. est.

H. reals

Planificació

01/04

09/04

4

5

Connectivitat i login

06/04

13/04

6

7

ER - Model Relacional

06/04

15/04

5

6

Esquema de seguretat

15/04

22/04

8

9

Bloc de manteniment

13/04

27/04

10

12

Alta disponibilitat

22/04

06/05

10

14

Bloc de consultes

27/04

09/05

8

8

Dummy Data

06/05

13/05

6

7

Exportació de dades

09/05

17/05

8

10

Document final instal·lació

13/05

20/05

4

5

Manual d'usuari

13/05

20/05

4

4

Document final

17/05

20/05

4

4

Total

77

91

Connexió a la base de dades: L'aplicació es connecta a PostgreSQL 16 des de Python utilitzant psycopg2-binary i SQLAlchemy 2.0. La cadena de connexió es carrega des de la variable d'entorn `===CODE===DATABASE_URL===ENDCODE===`.

Flux de login:

1. L'usuari introdueix usuari i contrasenya al client Tkinter
2. El client fa una petició `===CODE===POST /api/auth/login===ENDCODE===` amb les credencials
3. El servidor valida contra la taula `===CODE===APP_USERS===ENDCODE===` (contrasenyes hashejades amb bcrypt)
4. Si són correctes, retorna un JWT token amb el rol i el staff_id
5. El client emmagatzema el token en memòria (singleton de sessió)
6. Per a totes les peticions posteriors, s'envia com a `===CODE===Authorization: Bearer <token>===ENDCODE===`

Endpoints d'autenticació:

Mètode

Endpoint

Autenticació

Descripció

POST

No

Inici de sessió, retorna token + rol

POST

No

Crear nou usuari

GET

JWT

Llistar usuaris

PUT

JWT

Canviar pròpia contrasenya

PUT

JWT+ADMIN

Resetear contrasenya d'un altre usuari

PUT

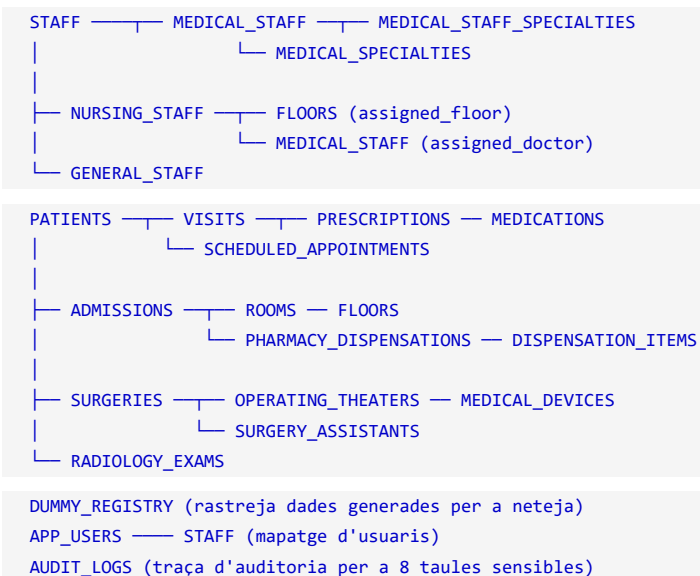
JWT+ADMIN

Activar/desactivar compte

Seguretat aplicada: Hashing bcrypt, tokens JWT signats amb clau de 256 bits, secrets en fitxer extern (===CODE===/etc/hms.env===ENDCODE===, permisos 600), comptes desactivables, 5 rols (ADMIN, DOCTOR, NURSE, STAFF, RECEPTIONIST).

La base de dades té 24 taules que cobreixen pacients, personal mèdic, visites, cirurgies, admissions, farmàcia, radiologia, auditoria i generació de dades de prova.

Model relacional:



Patrons de disseny: Base + subtipus (STAFF → MEDICAL_STAFF, NURSING_STAFF, GENERAL_STAFF), taules de junció, taula d'auditoria, taules de referència, capçalera-detall.

Convencions: Taules en majúscules snake case, columnes en minúscules snake case, PK `===CODE===<entitat>_id===ENDCODE===`, FK nom de la PK referenciada.

Principals taules: PATIENTS, STAFF, VISITS, SCHEDULED_APPOINTMENTS, SURGERIES, SURGERY_ASSISTANTS, ADMISSIONS, ROOMS, FLOORS, PHARMACY_DISPENSATIONS, DISPENSATION_ITEMS, RADIOLOGY_EXAMS, MEDICAL_SPECIALTIES, MEDICATIONS, OPERATING_THEATERS, MEDICAL_DEVICES, PRESCRIPTIONS, AUDIT_LOGS, APP_USERS, DUMMY_REGISTRY, MEDICAL_STAFF, NURSING_STAFF, GENERAL_STAFF, MEDICAL_STAFF_SPECIALTIES.

Matriu RBAC (5 rols PostgreSQL):

Rol

Límit connexions

Permisos

5

Accés complet a tot

50

CRUD a pacients, visites, prescripcions, cirurgies

100

L/E en camps limitats de pacients, admissions a la seva planta

10

CRUD a pacients, programar visites

30

Només lectura: nom del pacient + habitació (via vista `===CODE===patient_directory===ENDCODE===`)

Row-Level Security (RLS): Activat a PATIENTS i ADMISSIONS. Infermeres només veuen pacients ingressats a la seva planta. Variables de sessió (`===CODE===app.current\_user\_id===ENDCODE===`, `===CODE===app.current\_staff\_id===ENDCODE===`) establertes al login via `===CODE===SET_CONFIG===ENDCODE===`.

SSL: PostgreSQL configurat amb SSL utilitzant certificats autogenerats amb OpenSSL (validesa 1 any). `===CODE===pg_hba.conf===ENDCODE===` requereix `===CODE===hostssl===ENDCODE===` per a totes les connexions remotes.

Data masking: Control d'accés a nivell de columna per restringir dades sensibles (DNI, telèfon, email, adreça) segons el rol.

Auditoria: 8 triggers de BD registren automàticament totes les operacions INSERT/UPDATE/DELETE a taules sensibles (PATIENTS, VISITS, PRESCRIPTIONS, ADMISSIONS, SURGERIES, RADIOLOGY_EXAMS, PHARMACY_DISPENSATIONS, SCHEDULED_APPOINTMENTS) a la taula AUDIT_LOGS.

Compliment AGPD: Document complet preparat per a l'Agència de Protecció de Dades. Categories de risc: clínic (crític), identitat (alt), contacte (mig), laboral (mig), metadades d'auditoria (alt).

Operacions CRUD des de la pestanya Maintenance (5 subpestanyes: Doctor, Nursing, General Staff, Patient, Assignments):

Tasca

Endpoint API

Alta personal mèdic

Alta personal d'infermeria

Alta personal general

Alta nous pacients

Assignar infermer a metge/planta

Consultar cirurgies per data

Consultar visites per data

Consultar dispositius per quiròfan

Funcions/triggers PL/pgSQL (5 creades):

1. `===CODE===check_surgery_overlap()===ENDCODE===` — Evita reservar el mateix quiròfan al mateix dia/hora
2. `===CODE===validate_nurse_assignment()===ENDCODE===` — Comprova que el metge/planta assignat existeixi
3. `===CODE===audit_trigger_function()===ENDCODE===` — Registrador d'auditoria genèric per a 8 taules
4. `===CODE===get_current_app_user_id()===ENDCODE===` — Helper que retorna l'ID d'usuari actual
5. `===CODE===get_current_staff_id()===ENDCODE===` — Helper que retorna l'ID del treballador actual

Proposta de hardware:

Briar (primari): Intel Xeon E-2336, 16 GB RAM, 240 GB NVMe + 480 GB SATA SSD, LVM amb particions separades per a OS, logs, PGDATA i WAL.

Sion (standby): AWS EC2 t3.medium, 2 vCPU, 4 GB RAM, 80 GB gp3, LVM.

Topologia de replicació:

Mètode: Streaming replication nativa de PostgreSQL via WAL
Rol: Actiu-passiu (Briar = primary r/w, Sion = standby read-only)
Xarxa: Tailscale overlay network + LAN hospital (192.168.4.0/24)
Usuari de replicació: `===CODE===replicator===ENDCODE===`

Còpies de seguretat (Backup) - Sistema de 3 capes:

Capa

Patró de fitxer

Com es crea

Física (PGDATA)

Lògica (pg_dump)

per defecte o `===CODE===db-only===ENDCODE===`

Configuració

per defecte o `===CODE===config-only===ENDCODE===`

Script principal: `===CODE===scripts/backup_database.py===ENDCODE===` amb rotació local de 5 dies i pujada al núvol (Sion). Programació cron: backup complet diari a les 02:00, físic a les 03:00, freqüent cada 15 min.

Restauració: Script `===CODE===restore_service.sh===ENDCODE===` per a recuperació bare-metal completa. Ordre: PGDATA → config → dump lògic.

Informes accessibles des de les pestanyes Operational Reports, Statistics i Advanced Reports:

Obligatoris:

Dades d'una planta: habitacions, quiròfans i personal d'infermeria (`===CODE===GET /api/reports/summary?floor_id=X===ENDCODE===`)

Tot el personal: llistat complet (`===CODE===GET /api/reports/summary?report=staff===ENDCODE===`)

Visites per dia: nombre de visites ateses (`===CODE===GET /api/reports/visits?start_date=X&end_date=Y===ENDCODE===`)

Opcionals:

Ranking de metges: metges que atenen més pacients (`===CODE===GET /api/reports/doctor-workload===ENDCODE===`)

Top:

Malalties més comunes: diagnòstics més freqüents (`===CODE===GET /api/reports/summary?report=diseases===ENDCODE===`)

Reports complets: Summary, Visites, Cirurgies, Admissions, Medicacions. Exportació PDF via `===CODE===fpdf2===ENDCODE===`.

Dashboard API: `===CODE===GET /api/dashboard/stats===ENDCODE===` retorna visites avui, cirurgies avui, admissions actives, totals, visites per especialitat, tendència 7 dies, top 5 metges, admissions recents.

Generació de dades falses per a proves amb la llibreria Faker (locale `===CODE===es_ES===ENDCODE===` i `===CODE===ru_RU===ENDCODE===`). Ubicació:

`===CODE===server/src/app/services/dummy_service.py===ENDCODE===`.

Volums de dades:

Entitat	Objectiu	Generat
-----	-----	-----
Pacients	50.000	50.000
Visites	100.000	100.000+
Metges	100	100
Infermeres	200	200
Personal general	100	100
Cirurgies	~25.000	~25.000
Admissions	~25.000	~25.000

Procés: Dades de suport → Staff → Pacients → Visites (distribuïdes en 60 dies) → Cirurgies, admissions, prescripcions.

Ciríl·lic: 5% dels pacients i staff tenen noms en alfabet ciríl·lic (Faker locale `===CODE===ru_RU===ENDCODE===`).

Optimització: Insercions per lots amb `===CODE===flush()===ENDCODE===` cada 20 registres i `===CODE===commit()===ENDCODE===` cada 400. Temps reduït de ~20 min a ~5 min.

Neteja: Taula `===CODE===DummyRegistry===ENDCODE===` rastreja tots els registres. Opció "Cleanup dummy data" elimina tot en ordre invers.

Exportació XML/JSON: Les visites entre dues dates es poden exportar en format XML o JSON:

```
===CODE===GET /api/export/visits?start_date=X&end_date=Y&format=xml===ENDCODE===  
===CODE===GET /api/export/visits?start_date=X&end_date=Y&format=json===ENDCODE===
```

Inclou: identificador de visita, data, metge (nom + col·legiat), pacient (DNI, nom, cognoms, targeta sanitària).

Esquemes de validació: XSD i JSON Schema (`===CODE===server/src/app/schemas/===ENDCODE===`).

API Seguretat Social: `===CODE===POST /api/export/send===ENDCODE===` genera el fitxer i l'envia a l'API externa amb Basic Auth.

Dashboard:

Chart.js (web): `===CODE===http://localhost:5000/api/dashboard/view===ENDCODE===`
PowerBI (opcional): Connexió via `===CODE===http://localhost:5000/api/dashboard/stats===ENDCODE===`

Nota: Aquest manual està redactat en anglès a l'original, ja que és un document tècnic destinat a administradors de sistemes.

Requisits previs:

- Dos servidors Ubuntu 24.04 LTS (Briar i Sion) amb accés root
- Compte de Tailscale per connectar-los
- Python 3.12, PostgreSQL 16, Git

Passos resumits:

1. Instal·lar Ubuntu Server 24.04 amb particions LVM
2. Instal·lar i configurar Tailscale VPN
3. Configurar el firewall (UFW)
4. Instal·lar PostgreSQL 16 en tots dos nodes
5. Generar certificats TLS/SSL amb OpenSSL
6. Configurar PostgreSQL primari (Briar) i standby (Sion)
7. Configurar replicació streaming
8. Clonar el repositori i preparar l'entorn Python
9. Configurar variables d'entorn
10. Crear la base de dades i carregar esquemes
11. Configurar servei systemd
12. Instal·lar el client d'escriptori
13. Configurar còpies de seguretat automatitzades

14. Verificar el funcionament

Problemes trobats:

Problema

On

Solució

Briar

Canviar `===CODE===127.0.0.1/32===ENDCODE===` a `===CODE===192.168.4.0/24===ENDCODE===`

pg_basebackup timeout

Briar → Sion

Afegir `===CODE===sslmode=require===ENDCODE===` a `===CODE===primary_conninfo===ENDCODE===`

Tailscale no connectava

Sion (AWS)

Faltava port UDP 41641 al Security Group

Lag de replicació alt

Briar

psycopg2 no s'instal·lava

Briar

Tkinter no trobat

PC hospital

IDs reiniciats al restore

Briar

Canviar de pg_dump text pla a `===CODE===pg_dump -Fc===ENDCODE===`

Firewall bloquejava API

LAN Hospital

Obrir port 5000 amb equip de xarxa

Nota: Aquest manual està redactat en anglès a l'original.

Primers passos:

1. Iniciar el servidor API
2. Executar l'aplicació d'escriptori: `===CODE===python desktop/src/main.py===ENDCODE===`
3. Iniciar sessió amb les credencials

Seccions principals (barra lateral):

Dashboard — Resum del dia: visites, cirurgies, admissions actives

Maintenance — Alta de personal i pacients, gestió d'assignacions
 Data Workspace — Explorador de recursos: visualitzar, editar o eliminar qualsevol registre
 Operational Reports — Visites i cirurgies programades per data
 Statistics — Resum de planta, directori de personal, visites per dia, ranking de metges, malalties
 Advanced Reports — Visor multi-pestanya amb 8 categories i descàrrega PDF
 Dummy Data — Generar dades de prova (fins a 50.000 pacients)
 User Management (admin) — Crear usuaris, restablir contrasenyes
 Audit Logs (admin) — Visor de registres d'auditoria
 Change Password — Canviar contrasenya
 Logout — Tancar sessió

Rols del sistema:

Rol

Permisos

ADMIN

Accés complet, gestió d'usuaris i logs d'auditoria

DOCTOR

CRUD pacients, visites, tractaments, cirurgies

NURSE

Veure/editar pacients de la seva planta, admissions, assistir cirurgies

RECEPTIONIST

Registrar pacients, programar visites

STAFF

Només lectura: nom del pacient + habitació

Estratègia de backup (3 capes):

Física: PGDATA sencer (===CODE===/var/lib/postgresql/===ENDCODE===)

Lògica: ===CODE===pg_dump -Fc===ENDCODE=== (schema + dades)

Config: ===CODE===.env===ENDCODE===, systemd, logrotate,
 ===CODE===postgresql.conf===ENDCODE===, ===CODE===pg_hba.conf===ENDCODE===

Programació automàtica:

Hora	Tipus	Retenció
02:00 diari	Logical dump + config	5 dies
03:00 diari	Physical PGDATA	5 dies
Cada 15 min	Frequent logical	24 hores

Monitorització: Comprovar regularment l'estat de la replicació, la caducitat del certificat SSL, la salut de l'API i l'espai en disc.

This section is written in English.

Architecture: Three-tier system for Hospital de Blanes: Tkinter Desktop Client → Flask REST API → PostgreSQL 16.

What we built:

- 24 database tables with full referential integrity
- REST API with 24 endpoint groups (CRUD, reports, export, admin, audit, health)
- Tkinter desktop client with 12+ views
- RBAC with 5 roles (ADMIN, DOCTOR, NURSE, STAFF, RECEPTIONIST)
- Audit logging via PostgreSQL triggers (8 tables)
- SSL-secured connections
- Active-passive streaming replication (primary + cloud standby)
- Automated daily backups with 5-day retention
- XML/JSON export with XSD and JSON Schema validation
- Social Security API integration
- Chart.js web dashboard
- Dummy data generator with Faker (Cyrillic support, 50k patients, 100k visits)
- Health check endpoint (===CODE===GET /health===ENDCODE===)

Problems solved:

- Replication: pgpool-II was unstable, switched to native streaming replication
- Batch inserts: periodic commits fixed the 100k visit generation crash
- Audit triggers: SECURITY DEFINER was needed on helper functions
- RLS policies: nurses couldn't see their patients until floor assignment logic was fixed

Users and database role mapping:

Role	What they do	Database role
Administrator	Full access, user management, audit logs	===CODE===app_admin===ENDCODE===
Doctor	CRUD patients, visits, prescriptions, surgeries	===CODE===app_doctor===ENDCODE===
Nurse	Manage admissions, assist surgeries, view patients on their floor	===CODE===app_nurse===ENDCODE===
Receptionist	Register patients, schedule appointments	===CODE===app_receptionist===ENDCODE===
General staff	Read-only: patient name and room	===CODE===app_staff===ENDCODE===

External systems:

- Social Security API — receives monthly XML visit exports via HTTP POST (Basic Auth)
- AWS EC2 (standby) — hosts the standby PostgreSQL replica (Sion)

The database has 24 tables with full referential integrity. The schema is defined in ===CODE===scripts/sql/schema.sql===ENDCODE===.

Design patterns used: Base + subtype (STAFF → MEDICAL_STAFF / NURSING_STAFF / GENERAL_STAFF), junction tables, audit table, reference tables, header-detail.

Naming conventions: Tables in uppercase snake case, columns in lowercase snake case, PK ===CODE===<entity>_id===ENDCODE===, FK same as referenced PK.

Indexes: 85+ indexes on all foreign keys and frequently filtered columns (FKs, status, dates, timestamps).

Data dictionary main tables: PATIENTS (patient_id, national_id, names, birth_date, gender, contact, blood_type, allergies, health_card), STAFF (staff_id, national_id, names, staff_type, credentials), VISITS (visit_id, patient_id FK, doctor_id FK, visit_timestamp, diagnosis, notes).

RBAC: 5 PostgreSQL roles with granular table/column permissions defined in ===CODE===scripts/sql/security.sql===ENDCODE===.

Row-Level Security: Enabled on PATIENTS and ADMISSIONS. Nurses only see patients on their assigned floor. Session variables set via `===CODE===SET_CONFIG===ENDCODE===` in Flask login handler.

SSL: Self-signed certificates with OpenSSL, 1-year validity, auto-renewal check script (`===CODE===check_cert_expiry.sh===ENDCODE===`). All remote connections use `===CODE===hostssl===ENDCODE===`.

Data masking: Column-level grants restrict sensitive data per role. Doctors see all, nurses see limited columns (no `national_id`), general staff see only name + room via `===CODE===patient_directory===ENDCODE===` view.

Audit logging: 8 triggers on sensitive tables log to `AUDIT_LOGS` with user, timestamp, action, old/new data (JSON).

Initial issues: Forgot `===CODE===SECURITY DEFINER===ENDCODE===` on audit helpers, nurse RLS was too restrictive initially.

Data Controller: Hospital de Blanes, Blanes (Girona). Legal basis: LOPDGDD 3/2018 + GDPR (EU) 2016/679.

Risk classification:

HIGH (critical): Health data (diagnoses, surgeries, prescriptions, test results), genetic/biometric data

MEDIUM: Identity data (DNI/NIE, names, birth date), contact data, employment data, financial data

LOW: Internal user data (username, role), audit metadata

Technical measures: RBAC, bcrypt authentication, TLS encryption, RLS, data minimisation (views), audit triggers, concurrency control (surgery overlap check), encrypted backups, secret isolation (environment variables).

ARSULIPO rights: Access (patient history queries), Rectification (CRUD maintenance), Erasure (logical deletion), Restriction (RLS), Portability (XML/JSON export), Objection (administrative request).

Retention policy: Clinical history 15 years, employment 7 years, audit logs 2 years active + 7 compressed, backups 5 rotating daily + 12 weekly + 12 monthly.

Two-node setup:

Briar (on-premise): Intel Xeon E-2336, 16 GB RAM, 240 GB NVMe + 480 GB SATA SSD, LVM partitioned

Sion (AWS EC2): t3.medium, 2 vCPU, 4 GB RAM, 80 GB gp3

Replication: Native PostgreSQL streaming replication via WAL. Active-passive (Briar = primary r/w, Sion = standby read-only). Connected through Tailscale VPN.

Backup strategy (3 layers):

1. Physical - PGDATA via `pg_basebackup` (`===CODE===physical_*.tar.gz===ENDCODE===`)

2. Logical - `pg_dump -Fc` (`===CODE===hsp_db_*.dump===ENDCODE===`)

3. Config - `.env`, `systemd`, `postgresql.conf`, `pg_hba.conf`, `logrotate` (`===CODE===config_*.tar.gz===ENDCODE===`)

Automated schedule: Daily logical + config at 02:00, physical at 03:00, frequent every 15 min. Retention: 5 days local + optional rsync to Sion.

Restore operations: Logical restore (`===CODE===--restore latest===ENDCODE===`), config restore (`===CODE===--config-restore===ENDCODE===`), physical restore (`===CODE===--physical-restore===ENDCODE===`), full bare-metal (`===CODE===--full-restore===ENDCODE===`), and disaster recovery script (`===CODE===restore_service.sh===ENDCODE===`).

Failover: Promote Sion with `===CODE===pg_ctl promote===ENDCODE===`, then reconfigure API DATABASE_URL. Rebuild standby with pg_basebackup.

Monitoring: Replication lag (`===CODE===check_replication.sh===ENDCODE===`), SSL cert expiry (`===CODE===check_cert_expiry.sh===ENDCODE===`), disk space, audit logs, PostgreSQL logs.

Generated with Faker (es_ES locale, ru_RU for 5% Cyrillic). Generator in `===CODE===server/src/app/services/dummy_service.py===ENDCODE===`.

Target volumes: 50,000 patients, 100,000+ visits, 100 doctors, 200 nurses, 100 general staff, ~25,000 surgeries, ~25,000 admissions.

Process: Support data → Staff → Patients → Visits (60-day distribution) → Surgeries, admissions, prescriptions, dispensations, radiology.

Performance optimisation: Batch inserts with flush/commit, pre-computed diagnosis pools, random sampling. Cleanup via DummyRegistry table in reverse dependency order.

Login flow:

1. User enters credentials in Tkinter client
2. Client sends `===CODE===POST /api/auth/login===ENDCODE===`
3. Server validates against APP_USERS (bcrypt-hashed passwords)
4. On success: returns JWT token with role and staff_id
5. Token stored in memory session singleton
6. All subsequent requests include `===CODE===Authorization: Bearer <token>===ENDCODE===`

Roles: ADMIN, DOCTOR, NURSE, STAFF, RECEPTIONIST

User management endpoints: List, register, change password, reset password (admin), toggle active (admin). All passwords hashed with bcrypt, never stored in plaintext.

CRUD operations from the Maintenance tab: register medical/nursing/general staff, register patients, assign nurse to doctor/floor, view surgeries and visits by date, view medical devices by theater.

PL/pgSQL functions (5): check_surgery_overlap() trigger, validate_nurse_assignment() trigger, audit_trigger_function(), get_current_app_user_id(), get_current_staff_id().

Available reports (via API): Summary, Visits, Surgeries, Admissions, Medications, Financial, Radiology, Doctor Workload.

PDF export: Summary report via `===CODE===GET /api/reports/summary/pdf===ENDCODE===` (fpdf2). Requires JWT with ADMIN, DOCTOR, or NURSE role.

Desktop views: Operational Reports (daily visits/surgeries), Statistics (floor overview, staff directory, trends, rankings), Advanced Reports (multi-tab with 8 categories, filters, PDF download).

XML/JSON export: Visits between dates via `===CODE===GET /api/export/visits===ENDCODE===`. Includes visit ID, date, doctor name + license, patient DNI + name + health card. Validated against XSD and JSON Schema.

Social Security API: `===CODE===POST /api/export/send===ENDCODE===` generates XML/JSON, sends with Basic

Auth to external API.

Chart.js Dashboard: `===CODE===http://localhost:5000/api/dashboard/view===ENDCODE===` with visits today, surgeries today, active admissions, totals, specialty breakdown, 7-day trend, top doctors, recent admissions.

PowerBI (optional): Connect via `===CODE===http://localhost:5000/api/dashboard/stats===ENDCODE===` JSON endpoint.

Requirements: Two Ubuntu 24.04 LTS servers, Tailscale account, Python 3.12, PostgreSQL 16, Git.

Step-by-step summary:

1. Install Ubuntu with LVM partitioning (separate volumes for /, /var, /var/log, /tmp, PGDATA, WAL)
2. Tailscale VPN - connect Briar and Sion via mesh VPN
3. Firewall - UFW with port restrictions (22, 5000, 5432, Tailscale)
4. PostgreSQL 16 - install from official PostgreSQL APT repo
5. TLS certificates - generate CA + server certs with OpenSSL
6. Configure primary (Briar) - postgresql.conf with replication settings + pg_hba.conf with SSL
7. Configure standby (Sion) - postgresql.conf with primary_conninfo + standby.signal
8. Set up replication - create replicator user + slot on Briar, pg_basebackup on Sion
9. Clone repository - versioned deploy structure under /opt/hms
10. Environment variables - DATABASE_URL, JWT_SECRET_KEY, etc. in /etc/hms.env
11. Create database - run schema.sql, security.sql, initial_script.sql
12. systemd service - deploy_primary.sh + deploy.sh for production setup
13. Desktop client - install with Inno Setup or run from source

Verification checklist: API health endpoint, login, tables, replication, dashboard.

Server environment variables (on Briar): DATABASE_URL, JWT_SECRET_KEY, DB_NAME, EXTERNAL_API_URL/USERNAME/PASSWORD.

Desktop client: API_BASE_URL (default `http://192.168.4.254:5000/api`).

Ports: Flask API 5000, PostgreSQL 5432, Tailscale UDP 41641.

Production: Gunicorn (4 workers) behind systemd, logrotate, versioned deploy with automatic rollback.

Getting started: Start API server, launch desktop app with `===CODE===python desktop/src/main.py===ENDCODE===`, log in with credentials.

Sidebar sections: Dashboard (summary), Maintenance (staff/patient registration), Data Workspace (record browser), Operational Reports (daily visits/surgeries), Statistics (floor/trends/rankings), Advanced Reports (multi-tab viewer), Dummy Data (generate test data), User Management (admin only), Audit Logs (admin only).

Password: Change via sidebar. Logout: Click Logout in sidebar.

Roles: ADMIN (full access), DOCTOR (CRUD patients/visits), NURSE (floor-limited), RECEPTIONIST (register patients), STAFF (read-only).

Creating users: User Management → Create User (username, password, staff_id, role). Toggle Active to disable, Reset

Password for any user.

Backup strategy: 3 layers with automated schedule (daily logical, daily physical, frequent every 15min). Restore via `===CODE===backup_database.py===ENDCODE===` commands.

Monitoring: Replication status, SSL cert expiry, API health, disk space on both nodes.

Maintenance tasks: Rotate passwords periodically, check backup completion, review audit logs, archive old logs.

CAT: El projecte Hospital Management System és un sistema complet de gestió hospitalària desenvolupat per a l'Hospital de Blanes. Consta d'un client d'escriptori Tkinter, una API REST amb Flask i una base de dades PostgreSQL 16 amb replicació en streaming. S'han implementat totes les funcionalitats obligatòries, opcionals i top del projecte, incloent-hi gestió de pacients i personal, programació de visites i cirurgies, farmàcia, radiologia, consultes avançades, exportació de dades, dashboard web i seguretat integral (RBAC, RLS, SSL, auditoria).

ENG: The Hospital Management System project is a complete hospital management system developed for Hospital de Blanes. It consists of a Tkinter desktop client, a Flask REST API, and a PostgreSQL 16 database with streaming replication. All mandatory, optional, and top features have been implemented, including patient and staff management, visit and surgery scheduling, pharmacy, radiology, advanced queries, data export, web dashboard, and comprehensive security (RBAC, RLS, SSL, audit).

Tecnologia

Versió

Propòsit

Python

3.13

Backend

Flask

3.1

Web framework

Gunicorn

23.0

Production WSGI server

PostgreSQL

16

Database

SQLAlchemy

2.0

ORM

Tkinter

—

Desktop client

Chart.js

—

Web dashboard

Faker

40.15

Test data generation

Tailscale

—

VPN

JWT

4.7

Authentication

bcrypt

5.0

Password hashing

Àrea / Area

Assoliment / Achievement

Base de dades

24 taules, 85+ índexs, integritat referencial

API REST

24 grups d'endpoints

Client escriptori

12+ vistes d'usuari

Seguretat

RBAC (5 rols), RLS, SSL, bcrypt, JWT

Auditoria

8 triggers automàtics

Alta disponibilitat

Replicació streaming, failover, 3 capes de backup

Dades

50.000 pacients, 100.000 visites, 5% ciríl·lic

Exportació

XML/JSON, XSD/JSON Schema, API Seguretat Social

Dashboard

Chart.js, PowerBI opcional

Documentació

Guies d'instal·lació, usuari, administrador, AGPD